

# Conception de circuits numériques complexes sur FPGA SN423

---

**Louis Morge-Rollet**

Bureau : A203

[louis.morge-rollet@esisar.grenoble-inp.fr](mailto:louis.morge-rollet@esisar.grenoble-inp.fr)

(Ecrit par Vincent Beroulle et Louis Morge-Rollet)

# Responsable du cours

---

- [Louis Morge-Rollet](#), bureau A203
  - Maître de conférences Grenoble INP - Esisar
  - Enseignant à l'Esisar, Chercheur au LCIS
  - Ingénieur Grenoble INP - Esisar, Docteur en télécommunications ENSTA Bretagne
  - Travaux de recherche sur l'analyse de signaux radiofréquences dans un contexte embarqué

# I Introduction

---

# Informations préliminaires

## SN422

---

- Volumes horaires :
  - 6 séances de cours (+ 1 séance de CM/TD)
  - 5 séances de TD (Vincent Beroulle – Prof. Esisar)
  - 4 séances de TP (LMR et Mouad Rabiai – LCIS)
- Méthode d'évaluation :
  - Examen final, 2h, un seul document autorisé « VHDL Quick Reference Card », pas de calculatrice
    - note de l'examen de 2<sup>ème</sup> session (même condition) remplace la note de l'examen de 1<sup>er</sup> session seulement
  - Moyenne des notes de Contrôle Continu
    - 1 ou plusieurs Devoir(s) Surveillé(s)
    - QCMs réguliers
    - 1 compte rendu de TP (examen de l'an passé)

# Informations préliminaires

---

- Comptes rendus de TP notés mais pas corrigés
- Participation en TP prise en compte
- N'hésitez pas à poser des questions avant/pendant/après les TP à l'enseignant
- Délais max. de 3 semaines (ouvrées) entre le rendu des CR et la notation (donc retour des notes pas nécessairement avant l'examen)

# Informations préliminaires

## SN422

---

- 1 ECTS = 25 à 30h de travail total

Donc 3 ECTS = 75 à 90h

- SN423 : 37h présentiel

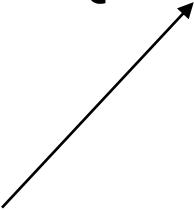
=> 38h à 53h de travail personnel

# Informations préliminaires

---

- Coefficients pour l'évaluation :
  - Examen **50%** + QCM **10%** + DS **15%** + TP **25%**

Voir slide suivant



Environ au milieu du cours



- Rattrapage :

$$N2 = \text{Max}(E2, 0.5 \times E1 + 0.25 \times TP + 0.15 \times DS1 + 0.1 \times QCM1)$$

# QCM

---

- Les QCM sont à **faire régulièrement : avant la fin de la semaine suivant le cours**
- Certains QCM comptent pour la note finale (Examen), d'autres sont des QCM d'auto-évaluation
- La durée pour répondre aux QCM est limitée (et variable)
- Le nombre de questions (tirées aléatoirement) varie selon le QCM



# QCM

---

SN360/SN361 (Auto-évaluation) QCM0

SN360/SN361 QCM1

SN423 Chap I QCM1

SN423 Chap II QCM2

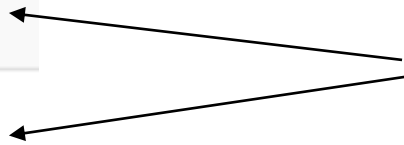
SN423 Chap III QCM3

SN423 Chap IV QCM4

SN423 Chap V QCM5

SN423 Tous les chapitres QCM6

**A FAIRE**



# Informations préliminaires

---

- Objectif du cours :

**Savoir concevoir en VHDL RTL un système matériel numérique complexe à base de composants reconfigurables**

- Prérequis : conception logique, machine à états finis de Moore et de Mealy, VHDL

# Informations préliminaires

---

- Chamilo :
  - Diapositives du cours
  - *VHDL quick reference card*
  - Sujets de TD et de TP
  - Examens avec corrections
  - **Vidéos**
  - **QCM**

# Informations préliminaires

## Bibliographie

---

- **RTL Hardware design using VHDL: coding for efficiency, portability and scalability, Pong. P. CHU, WILEY INTERSCIENCE, 2006. ISBN 0-471-72092-5**
- **FPGA Prototyping by VHDL examples: Xilinx Spartan™-3 Version, Pong. P. CHU, WILEY INTERSCIENCE, 2008, ISBN 978-0-470-18531-5**
- **The Zynq book: Embedded Processing with the ARM® Cortex®-A9 on the Xilinx® Zynq®-7000 All Programmable SoC, Louise H. Crockett et al. ([lien](#))**

# Informations préliminaires

---

- **VHDL, du langage au circuit, du circuit au langage, J. Weber, M. Meaudre, Masson**
- Circuits numériques et synthèse logique, un outil: VHDL, J.Weber, M. Meaudre, Masson (621.395 WEB)
- Initiation au langage VHDL, Michel Aumiaux, Dunod
- VHDL, langage, modélisation, synthèse, Airiau, Bergé, Olive, Rouillard, P. P. Romandes
- VHDL, Introduction à la synthèse logique, P. Larcher, Eyrolles

# Informations préliminaires

---

- Outil de synthèse et simulateur VHDL (Vivado WebPACK) disponible gratuitement
  - [sur le site Xilinx](#)
  - Installation lourde!
- Ces outils sont installés dans toutes les salles de TP

# I Introduction

## Plan global du cours

---

- **I Introduction**
  - Rappels : **QCM0 + QCM1**
  - Quand utiliser un composant reprogrammable?
  - Synthèse RTL
- II Conception de systèmes synchrones
- III Synthèse RTL
- IV Synthèse de haut niveau (HLS)
- V Conception de filtres

# I Introduction

## Rappel : Contexte & Problématique

---

- Actuellement, les outils de *synthèse logique* (les plus répandues) font la synthèse du niveau RTL vers le niveau porte
    - Les instructions utilisées doivent être synthétisables
    - Restrictions très fortes sur le style d'écriture VHDL
- s <= '1' after 10 ns; -- pas synthétisable/simulable
- wait on s1, s2; -- synthétisable/simulable
- wait for 10 ns; -- pas synthétisable/simulable



# I Introduction

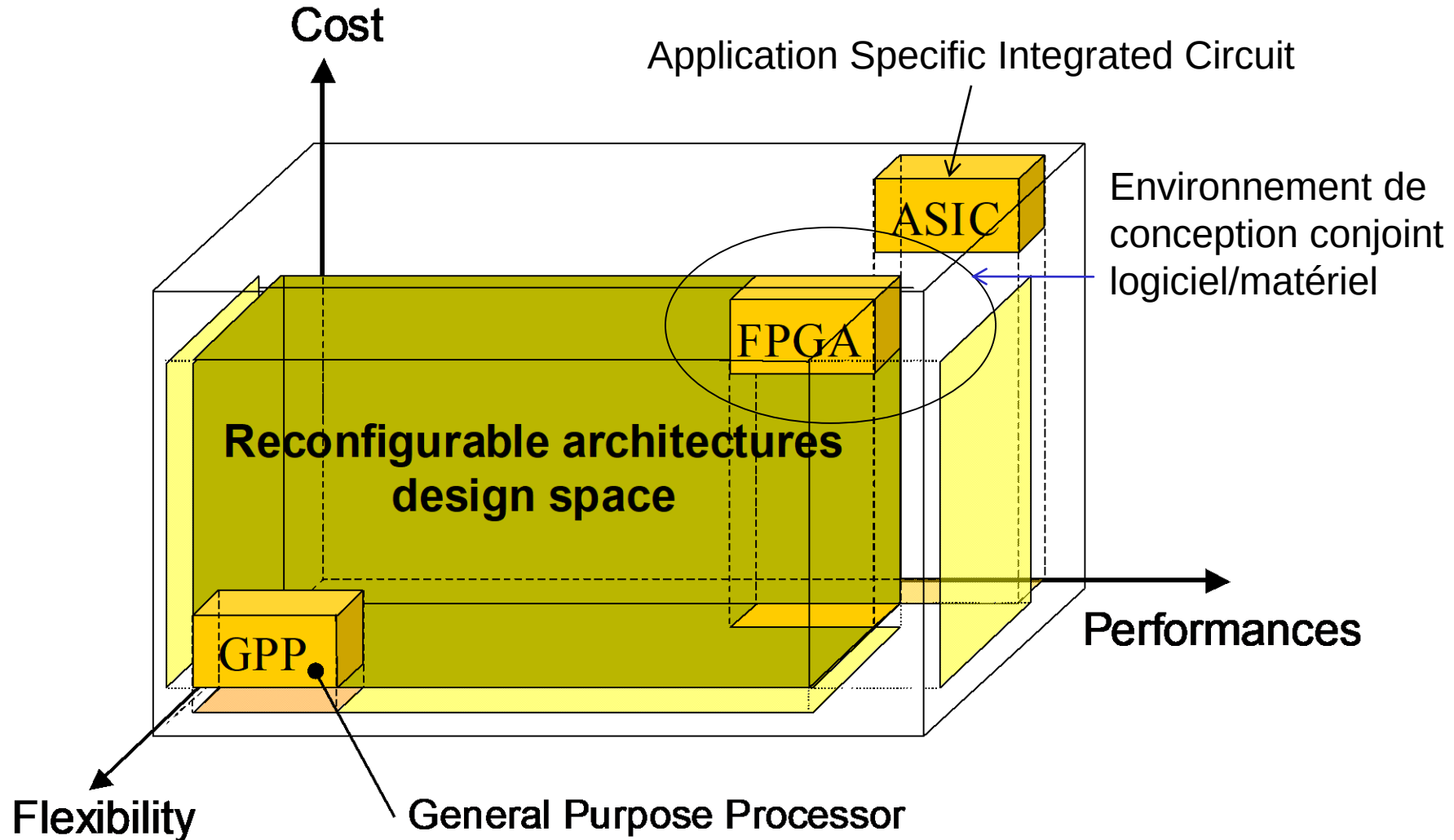
Rappel : Quand utiliser un composant reprogrammable?

---

- Deux architectures existent :
  - Les architectures programmées (micro-processeur, microcontrôleur, digital signal processor...) :
    - Une même ressource (ex. UAL) permet de réaliser plusieurs fonctions grâce à un découpage séquentiel du traitement (programme séquentiel)
    - Grande flexibilité mais vitesse limitée (plusieurs cycles par instruction)
  - Les architectures câblées (PAL, CPLD, FPGA, ASIC) :
    - Traitement en parallèle des tâches (concurrency)
    - Faible flexibilité (en particulier pour les composants non reprogrammables) mais grande rapidité d'exécution :
      - Traitement de données en un cycle d'horloge (Bus PCI, traitement d'images...)

# I Introduction

Rappel : Quand utiliser un composant reprogrammable?



# I Introduction

## Quand utiliser un composant reprogrammable?

---

- SoPC : *System on Programmable Chip*
  - Utilisation de processeurs dans les FPGA (Intel, ARM)
  - De nombreuses IP (*Intellectual Properties*) de processeurs
    - Composants câblés (*hard* IP) ou programmés (*soft* IP) : <http://opencores.org/>
    - L'approche purement RTL est délaissée pour les systèmes complexes
- eFPGA (embedded FPGA): implanter des blocks FPGA dans des processeurs

# I Introduction

Quand utiliser un composant reprogrammable?

---

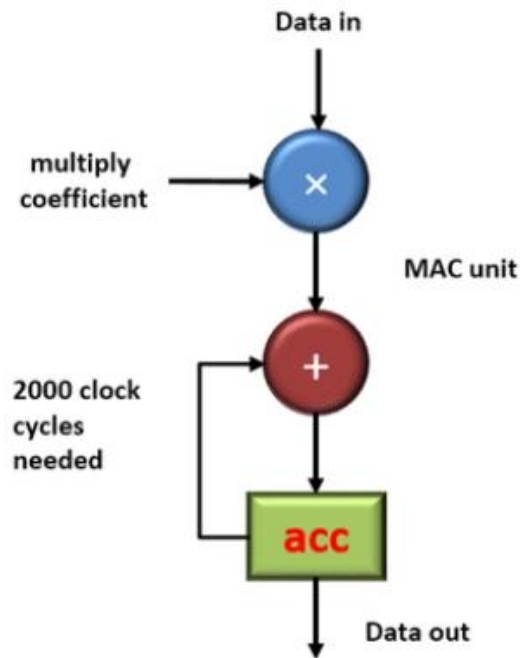
*Pour le traitement du signal : Digital Signal Processors (DSP) ou FPGA?*

- les FPGA récents intègrent de nombreux blocs de « multiplication accumulation » (MAC) très performants
- Matlab ou d'autres outils permettent de **générer automatiquement du code VHDL** pour les FPGA (**comme ils génèrent du code C pour les DSP**)

# I Introduction

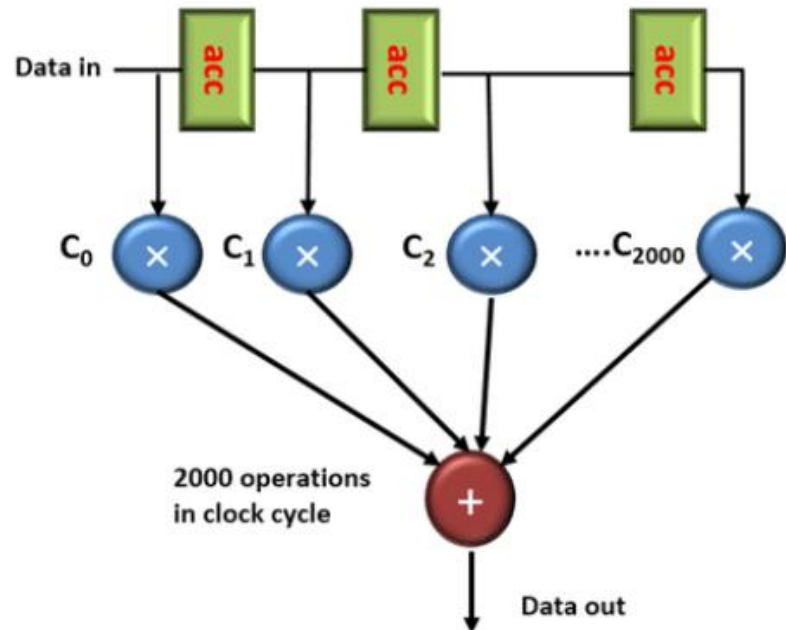
Quand utiliser un composant reprogrammable?

**Standard DSP processor  
(sequential implementation)**



$$\frac{2 \text{ GHz}}{2000 \text{ clock cycles}} = 1000 \text{ KSPS}$$

**FPGA implementation  
(completely parallel design)**



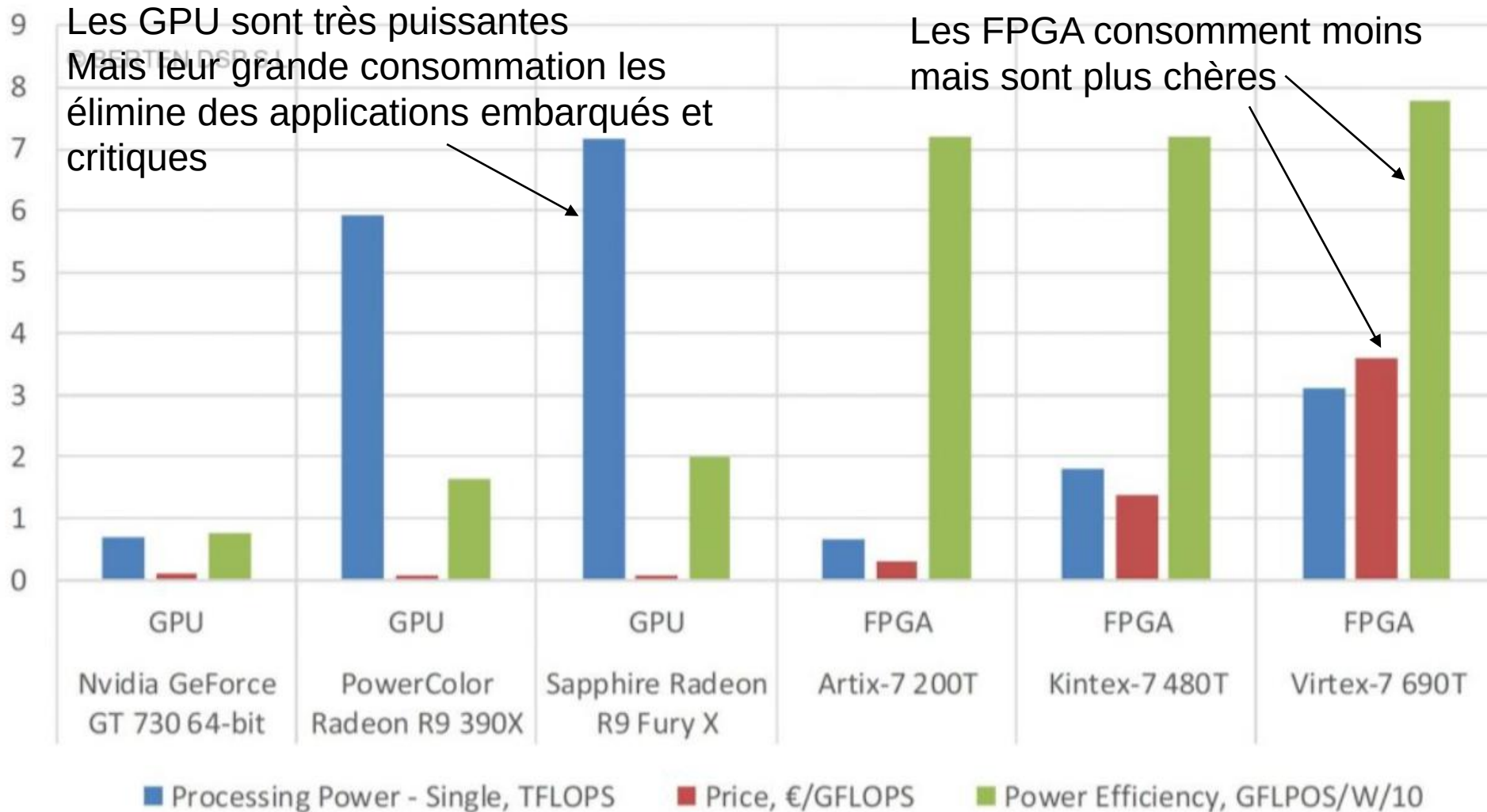
$$\frac{600 \text{ MHz}}{1 \text{ clock cycles}} = 600 \text{ MSPS}$$

# I Introduction

## Quand utiliser un composant reprogrammable?

Les GPU sont très puissantes  
Mais leur grande consommation les élimine des applications embarqués et critiques

Les FPGA consomment moins  
mais sont plus chères

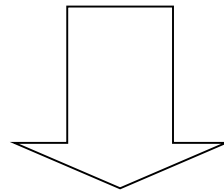


# Introduction

## Synthèse RTL

---

- L'optimisation réalisée par l'outil de synthèse logique porte uniquement sur **l'optimisation des équations combinatoires obtenues à partir de descriptions RTL de systèmes.**
- Comment obtenir des descriptions RTL optimales pour un cahier des charges donné ?



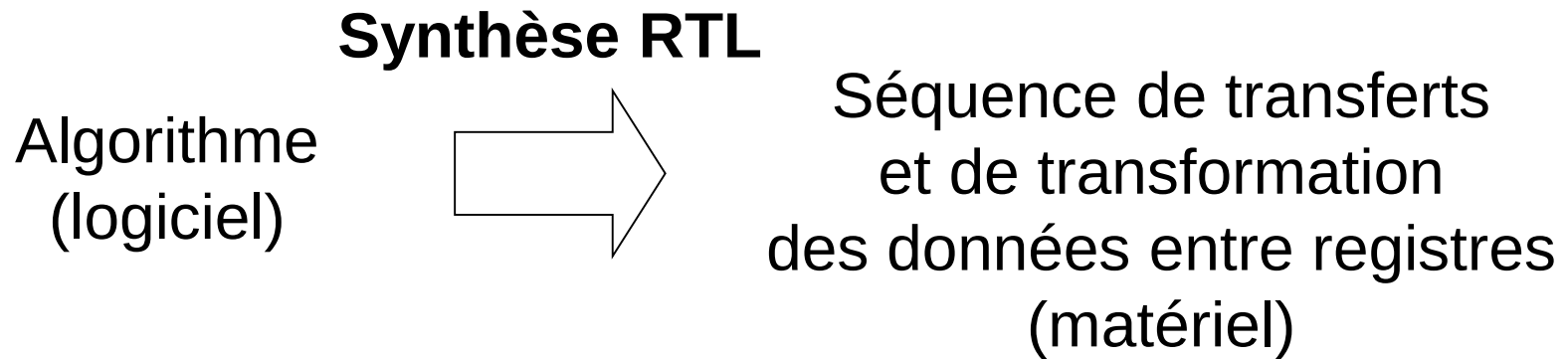
## Méthodes de synthèse RTL

# Introduction

## Synthèse RTL

---

- Transformation qui, à partir d'une *spécification d'un* algorithmique ***génère une description au niveau RTL***





# Introduction

## HLS

---

- Les outils de synthèse haut niveau (HLS: *High Level Synthesis*) s'améliorent rapidement (Catapult HLS, Vivado HLx, Simulink HDL, Cadence Stratus)
  - Transforment les algo. en code RTL
  - Permettent un gain de productivité de x6
  - Accessibles aux développeurs logiciels (C/C++)
  - Restent inférieurs en terme de qualité des résultats

# Introduction

## Plan global du cours

---

- **I Introduction**
- II Conception de systèmes synchrones
- III Synthèse RTL
- IV Synthèse de haut niveau (HLS)
- V Conception de filtres

- 
- **Faire le QCM associé à ce chapitre sur Chamilo**